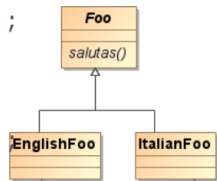


Lezione 5 23/10/23 Binding + interfacce

giovedì 26 ottobre 2023 12:02

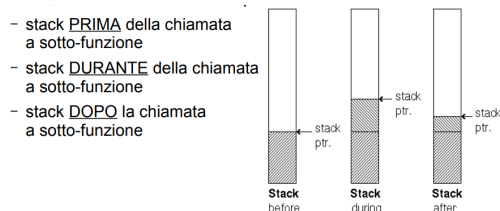
Ritorniamo all'aspetto "dinamico" di java : java soffre del problema della ricompilazione (**Constant Recompilation problem**) , in quanto java non usa i spiazamenti add offset , ma usa degli spiazamenti simbolici . Questo problema in java viene riferito come **Java Fragile Base Class Problem** : ogni volta che modifico una classe, tutte quelle cui riferisce, devono essere ricompilate. Torniamo ad esempio di foo:



In questo caso in base a test la variabile sarà di un tipo piuttosto che altro ! . Nota : f è variabile di tipo Foo , mentre il tipo di istanza di f dipende dalla condizione del test. Questo viene fatto grazie al **binding** : associazione tra invocazione di operazione con effettivo metodo da invocare . Due tipi :

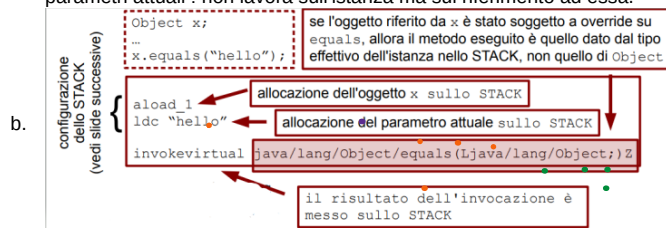
1. **Early binding (static binding)** : associazione viene fatta a tempo di compilazione :
 - a. Quindi di basa su offset numerici
 - b. Fatto sul tipo della variabile
2. **Late binding (dynamyc binding)**: associazione fatta a run-time :
 - a. Viene fatto al momento in cui invoco operazione (messaggio dove va??) , quindi a tempo di esecuzione . Questo tipo di associazione viene fatta sul tipo dell'istanza dell'oggetto.
 - b. Il compilatore non sa quale è il tipo dell'oggetto : quindi questo collegamento viene effettuato a run-time , usando meccanismi a run-time che consentono di recuperare il tipo dell'istanza e crea legame tra metodo ed operazione .
 - c. In java solo questo tipo di approccio : tranne nel caso di metodi dichiarati static , final e private .
 - i. Static: nel primo caso si hanno operazioni in ambito di classe : invoco operazioni attraverso in nome della classe e non dell'istanza , quindi per qualunque istanza l'operazione viene svolta nello stesso modo . Riferimento del metodo noto a tempo di compilazione
 - ii. Final : parola chiave che se applicata ad un metodo, quel metodo non può essere sottoposto ad override (ridefinizione nel figlio del metodo ereditato dal padre) . Disabilito in binding dinamico.
 - iii. Private : attributi e operazioni sono nella classe : quindi ammesso il binding dinamico lo risolvo mi riferisco ad istanza della classe stessa . Da notare che in java i metodi private sono implicitamente final.

Andiamo a vedere come funziona il binding dinamico : ma prima ricordiamoci stack e heap : il primo riferisce ad un'aria di memoria riservata per la computazione di un'applicazione e ve ne è uno per ogni thread in esecuzione (variabili locali, riferimenti ecc ecc) , mentre per il secondo viene usato per riferire alla memoria dinamica: dati dell'applicazione , i quali sono globalmente accessibili e quest'area di memoria viene gestita dal programmatore : vi è un solo heap condiviso per tutti i thread (allocazione nuove istanze) . Deallocazione implicita del gc . In dettaglio :

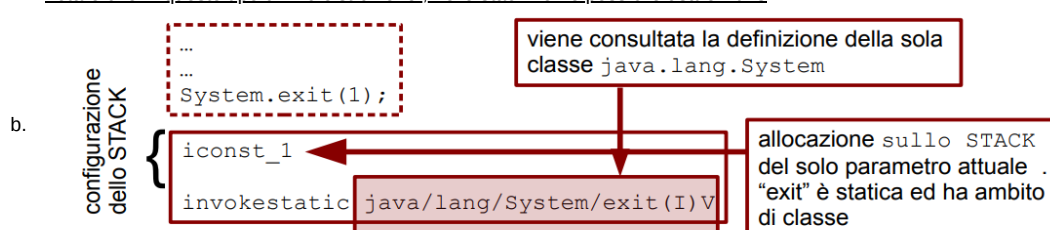


Per ogni classe istanziata vi è il **Java Runtime Constant Pool** : are di memoria dove vengono messe le costanti, riferimenti a tempo di compilazione , riferimenti definiti a run-time. Come vengono risolti questi riferimenti ? Attraverso 3 istruzioni nell'assembly della jvm (istruzione bytecode) :

1. **Invoke virtual** : dinamico
 - a. Viene constatato il run-time constant pool accedendo in base a tipo dell'oggetto corrente (sullo stack) ed usa i parametri attuali : non lavora sull'istanza ma sul riferimento ad essa.

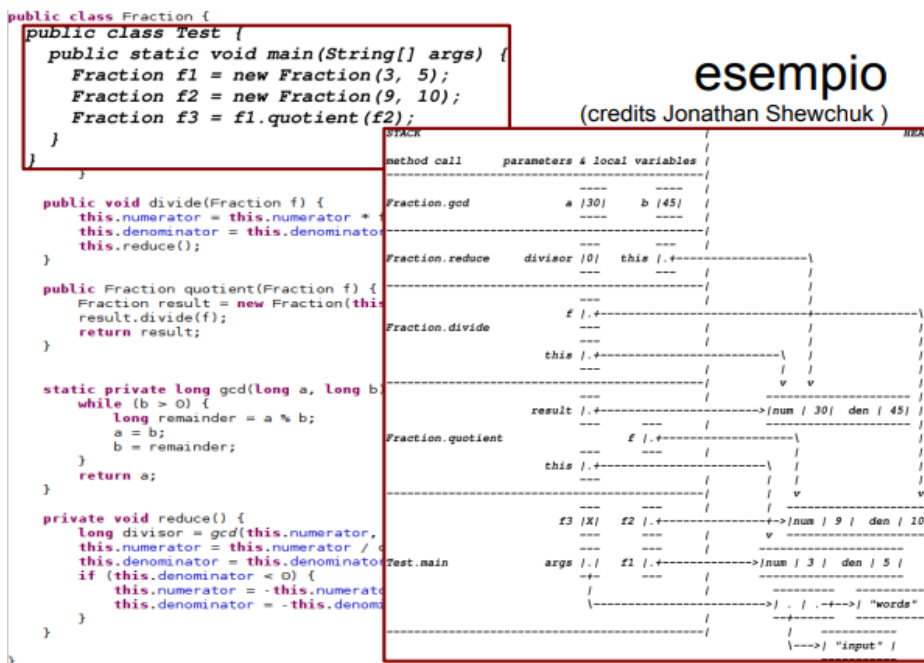


2. **Invoke static** : statico
 - a. Viene constatato il run-time constant pool in base il base alla classe referenziata ed usando i parametri formali . Da notare che in questo tipo di microistruzione , nello stack non è possibile usare il this .

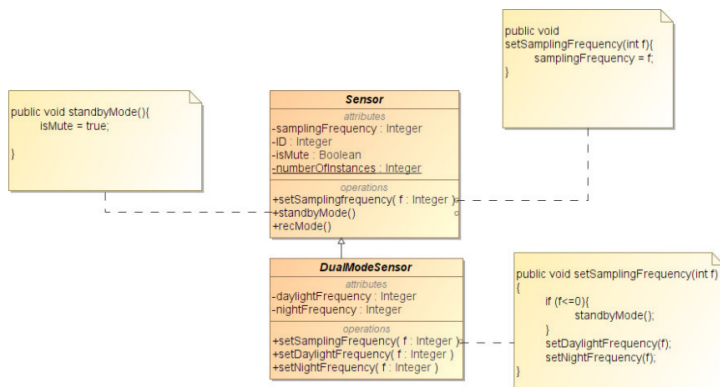


3. **Invoke special** : ibrido : usato per gestire il main , metodi private e final
 - a. viene consultato il run-time constant pool in basse alla classe referenziata , usando i parametri attuali (invoke static) , inoltre mantiene riferimento ad istanza corrente attraverso il this (invoke virtual).

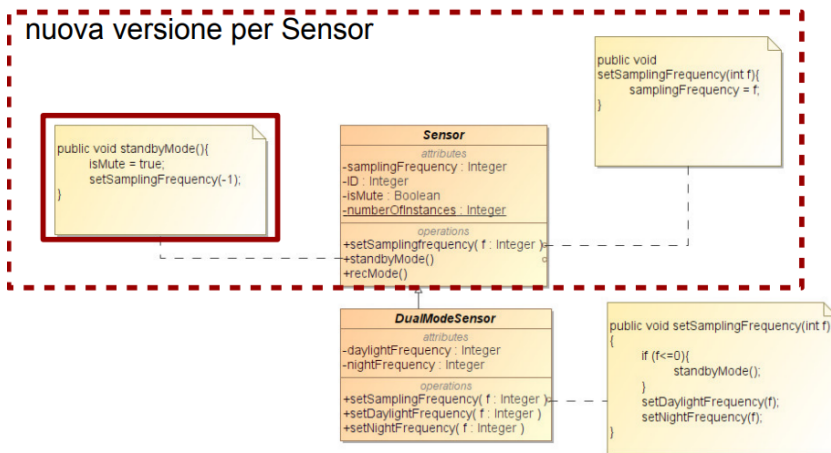
Vediamo un esempio delle chiamate di queste micro istruzioni :



Torniamo ora all'esempio del sensore e contestualizziamo il FBCP:



Ma che succede se il metodo associato un'operazione cambia ?



Se applico questo snippet , viene usato il metodo della sottoclasse:

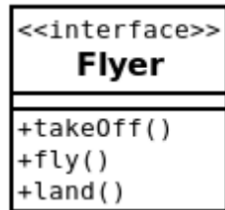
```

DualModeSensor s;

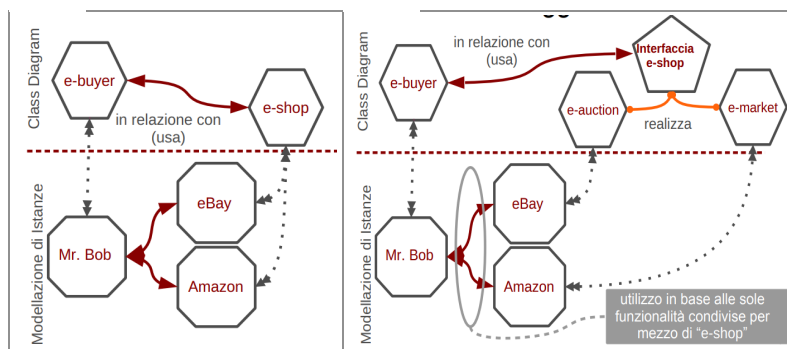
s.setSamplingFrequency(-100);

```

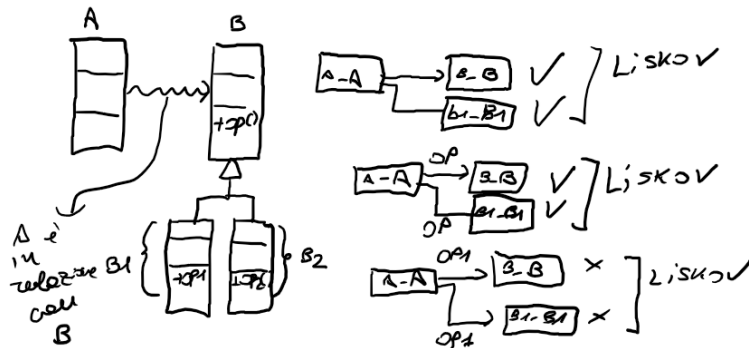
Quindi in generale : se il padre evolve nel tempo , potrebbe implicare che non tiene conto di override ed overloading della classe figlio , quindi ogni volta che "cambio" il padre devo chiedere ai figli "questo cambiamento provoca danni?" : quindi bisogna vedere il come del padre va incastrato con quella del figlio . Si deve quindi mitigare il coupling (accoppiamento tra le classi) . In generale questo problema non si risolve : al massimo si può mitigare . Andiamo a vedere le **interfacce (specifiche parziali del sistema ed inter collegano sistemi diversi)** : collezioni di operazioni che sono utilizzate per specificare un servizio di una classe o componente. Definisce solo la segnatura delle operazioni e non può essere istanziata . Dichiara ed enfatizza i contratti e non ha particolare implementazione . Definite in java, in uml mentre in c++ no. Piccolo hint . Da java 8 si possono definire operazioni di default. In uml viene rappresentata così :



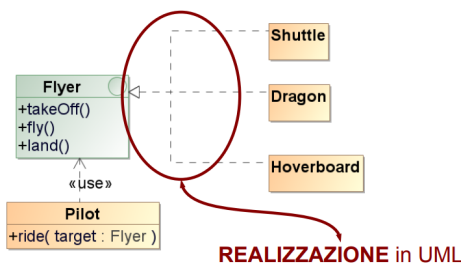
Quindi nel nostro caso l'oggetto volante rappresenta qualunque cosa che riesca ad interagirci attraverso le operazioni . Facciamo ora un paragone : **interfaccia vs classi astratte** : entrambe modellano operazione associate a nessun metodo , impongono override alle sottoclassi , entrambi specificano solo segnatura delle operazioni e nessuna può essere istanziata . Quindi interfaccia modella gli schemi di interazione (aumentando disaccoppiamento tra cosa e come) ed **ignora il polimorfismo** : riassumendo è una vista del sistema , mentre le seconde modellano una parte della struttura statica del sistema e possono definire attributi. **Nota : che se in una classe abstract tutte le operazioni sono astratte si potrebbe considerare come una interfaccia.** Quindi per mitigare il fbcip si usano le interfacce (vengono esportate) : si aumenta la modularità , riuso ed implementazione : quindi si controlla evoluzione padre-figlio . Torniamo ad esempio e-buyer / e-shop :



Quindi in generale :



Torniamo all'esempio dell'interfaccia :



```

public interface Flyer{
    /** Dichiarazione
     * dell'interfaccia
     */
    public void takeOff();
    public void fly();
    public void land();
}

public class Dragon implements Flyer{
    /** Dichiarazione
     * dei metodi per l'interfaccia
     * e corpo della classe
     */
}
  
```

Vediamo ora quando conviene usare ereditarietà o realizzazione : nel primo caso le caratteristiche comuni vengono definite nella classe (cosa + come), vi è una relazione di is-a-kind-of tra le classi , aumenta accoppiamento quindi ma introduce FBCP , quindi si arriva ad una forma di incapsulamento più debole , mentre nella seconda c'è il riuso delle operazioni pubbliche definite dall'interfaccia , quindi si accettano i contratti senza accettare i vincoli sui dettagli implementativi . Nota : java non supporta ereditarietà multipla , ma una classe può implementare più interfacce contemporaneamente (vista = insieme operazioni definite da interfaccia) . Attenzione se ereditarietà e realizzazione.

```
public interface Flyer {
    public void fly();
    public void takeOff();
    public void land();
}

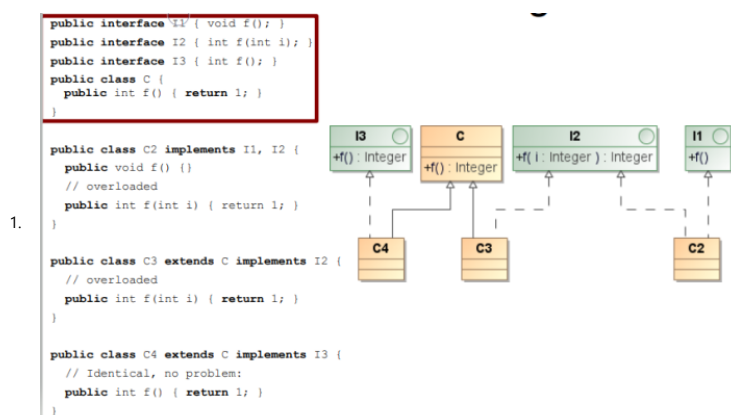
public class SuperHero implements
    Flyer, Fighter, Swimmer {
    public void fly() { ... }
    public void takeOff() { ... }
    public void fight();
    public void land() { ... }
    public void fight() { ... }
    public void swim() { ... }
}

public interface Fighter {
    public void takeOff() { ... }
    public void land() { ... }
}

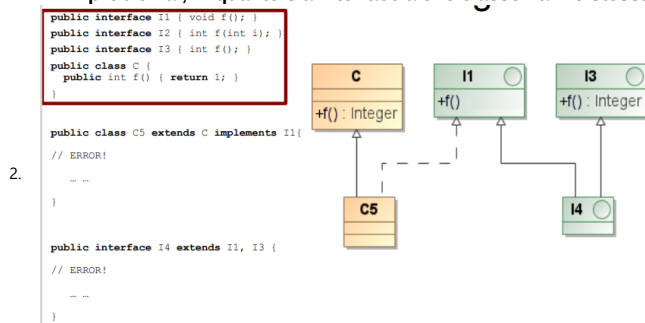
public interface Swimmer {
    public void swim();
}
```

35 di 47

Vediamo degli esempi :

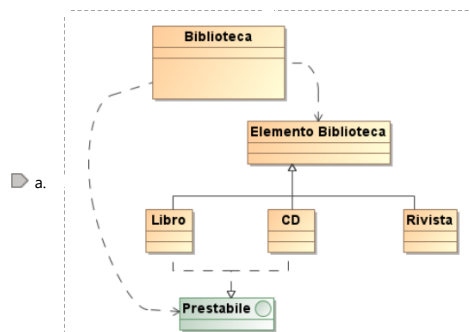


a. In questo esempio notiamo che nella classe C4 , nonostante realizza I3 (interfaccia) ed estende C non vi è problema , in quanto sia interfaccia che classe hanno stessa segnatura /firma dell'operazione f(): Integer.



a. Qui invece errore perché la jvm non distingue i tipi di ritorno dell'operazione . Mentre in UML due operazioni !

3. modellare un semplice sistema di gestione per una biblioteca. Considerare che la biblioteca è in grado di gestire il prestito di *almeno* libri e CD



b. Ricordiamoci qui che il modello dipende dalla soluzione che si vuole avere

